

兆鸣嵌入式

C语言·一个LED讲透面向对象

EP08 你用C手搓了一个class

三家世界500强·11年一线嵌入式实战

公众号搜索「兆鸣嵌入式」回复「交流」进群

核心概念

C语言没有class关键字，但你天天都在写class：

函数前缀 = 类名，init = 构造函数，deinit = 析构函数，.h + .c = 一个完整的类。

C语言没有class？你天天都在写

设计思路

问题1：名称冲突

LED模块和电机模块都有init()、on()、off()——编译器报错“名称冲突”。两家店都叫“init”，快递员不知道送哪家。

问题2：调用顺序混乱

实习生不看文档，没调init()直接用led_on()——GPIO没配置就写电平，硬件行为不可预测。

解决：前缀=类名，init/deinit=构造/析构，.h+.c=一个类

1. led_前缀 = LED类名，motor_前缀 = Motor类名——看到前缀就知道是谁家的。
2. init做三件事（参数验证+硬件配置+默认状态），就是构造函数。
3. deinit做两件事（关闭硬件+释放资源），就是析构函数。
4. 一个.h + 一个.c = 一个完整的C语言“类”。

视频讲义

项目越做越大——函数名"打架"了

两个模块，同名函数，编译报错

led.c LED模块

你写的LED驱动

```
void init(Led_t *me);
void on(Led_t *me);
void off(Led_t *me);
```

VS

motor.c 电机模块

新加的电机驱动

```
void init(Motor_t *me);
void on(Motor_t *me);
void off(Motor_t *me);
```



error: multiple definition of 'init'

C语言规则：所有非Static函数，名字必须全局唯一

兆鸣嵌入式

名称冲突：两个模块都叫init，编译器报错

前缀 = 品牌名

每个模块的函数加上模块名前缀，清清楚楚

led.c LED模块

所有函数以 led_ 开头

```
void led_init(Led_t *me);
void led_on(Led_t *me);
void led_off(Led_t *me);
```



motor.c 电机模块

所有函数以 motor_ 开头

```
void motor_init(Motor_t *me);
void motor_on(Motor_t *me);
void motor_off(Motor_t *me);
```



沙县拌面 沙县蒸饺 兰州牛肉面 兰州凉皮

前缀 = 品牌名 一看就知道是谁家的

兆鸣嵌入式

前缀命名：led_init vs motor_init，永远不冲突

init = 开门营业 deinit = 打烊

一个模块的完整生命周期

led.h 完整接口

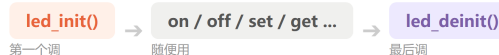
一个模块对外的全部“菜单”

/* LED 模块公共接口 */

```
int led_init(Led_t *me, uint8_t pin);
void led_on(Led_t *me);
void led_off(Led_t *me);
void led_set_brightness(Led_t *me, uint8_t val);
uint8_t led_get_state(Led_t *me);
void led_deinit(Led_t *me);
```

led_init 做了什么？（开门营业）

- 1 检查参数 — 引脚号合不合法
- 2 配置硬件 — 设置GPIO口工作模式
- 3 设默认值 — 灯默认关闭，状态归零



init 永远第一个 deinit 永远最后一个 中间随便用

兆鸣嵌入式

init/deinit生命周期：开门营业→服务→打烊收工

C语言 vs C++：命名与生命周期

C 手动写前缀和 init, C++ 交给编译器

C语言 你手动写

前缀 + init/deinit

```
/* 前缀 = 命名空间 */
led_init()
led_on()
led_off()
```

/* 构造：手动调 */

```
led_init(me, pin);
```

/* 析构：手动调 */

```
led_deinit(me);
```

=

底层相同

C++ 编译器帮你写

类名 + 构造函数/析构函数

```
/* 类名 = 命名空间 */
Led::init()
Led::on()
Led::off()
```

/* 构造：自动调 */

```
Led led(pin); // 自动调构造
```

/* 析构：自动调 */

```
~Led() { /* 关GPIO */ }
```

struct + me = class + this ✓

static + .h/.c = private + public ✓

前缀 + init/deinit = 类名 + 构造/析构 ✓

恭喜你，一个完整的 C++ 类的核心机制，你已经全懂了

兆鸣嵌入式

C vs C++ 对照：前缀+init/deinit = 类名+构造/析构

代码结构

文件	说明
led.h	LED类的公共接口——led_前缀 + initialized标志
led.c	LED类的实现——init三件事 + 引脚范围检查
motor.h	Motor类的公共接口——motor_前缀，与LED和平共处
motor.c	Motor类的实现——结构与LED完全一样
main.c	演示两个"类"和平共处 + 不调init的后果
Makefile	编译脚本

关键代码解读

1. 两个模块和平共处——前缀就是类名

main.c

```
#include "led.h"
#include "motor.h"

Led_t red_led;
Motor_t fan_motor;

led_init(&red_led, 5);    /* LED类的构造函数 */
motor_init(&fan_motor, 8); /* Motor类的构造函数 */

led_on(&red_led);         /* LED类的方法 */
motor_start(&fan_motor);  /* Motor类的方法 */
```

led_和motor_前缀不同，永远不冲突。就像"沙县小吃"和"兰州拉面"都卖汤，但你不会走错门。

2. init三件事——构造函数的标准动作

led.c

```
int led_init(Led_t *me, uint8_t pin)
{
    /* 第一件事：参数验证 */
    if (me == NULL) return -1;
    if (!is_pin_valid(pin)) return -2;

    /* 第二件事：硬件配置 */
    platform_gpio_init(pin, GPIO_MODE_OUTPUT);

    /* 第三件事：设置默认状态 */
    me->pin = pin;
    me->brightness = 0;
    me->is_on = false;
    me->initialized = true;

    update_hardware(me);
    return 0;
}
```

3. 没调init直接用——返回错误码而不是崩溃

main.c

```
Motor_t new_motor = {0}; /* 没调motor_init */
int ret = motor_start(&new_motor);
/* 返回 -3 (未初始化), 不会崩溃 */
```

initialized标志位防止未初始化就使用。没开门营业，你不能进店吃饭。

C vs C++ 对照

C 写法	C++ 写法
led_ 前缀	Led:: 作用域
led_init()	Led::Led() (构造函数)
led_deinit()	Led::~~Led() (析构函数)
motor_ 前缀	Motor:: 作用域
一个.h + 一个.c	一个class定义
initialized标志位	RAII自动管理

编译运行

```
cd code/EP08_手搓class  
make  
./demo
```

获取更多

公众号搜索「兆鸣嵌入式」

回复「交流」加群，与一线工程师切磋

抖音 / B站 / 视频号

搜索「兆鸣嵌入式」